

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1108

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4,BL5)
Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:35

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E. Ragul

Annexure A

Constraint-Based Problem Solving

Q1. Scenario: Online Banking System

System Components:

Account Management, Fund Transfer, Bill Payment, Transaction History

Given Constraints:

- Transaction completion time < 2 seconds
- Multi-factor authentication required
- Support 5000+ concurrent users

Question:

Analyze the system using quality assurance techniques under the given constraints. Identify potential defects related to security, performance, and reliability. Propose suitable testing strategies and justify how they ensure system quality.

Q2. Scenario: E-Commerce Shopping Application

System Features:

Product Search, Shopping Cart, Checkout, Payment Gateway

Usability Constraints:

- Checkout process within 3 steps
- Easy product navigation
- Responsive design for mobile and desktop devices

Question:

Apply usability testing techniques to evaluate the application. Identify usability challenges and recommend design improvements that satisfy the given constraints. Explain how the proposed changes improve customer satisfaction and efficiency.

Q3. Scenario: Hospital Management System

Modules:

Patient Registration, Appointment Scheduling, Medical Records, Billing

QA Constraints:

- Patient data accuracy must be maintained
- System availability 24/7

- Simultaneous access by multiple departments

Question:

Use quality assurance practices to analyze the system. Identify critical test cases and discuss how the constraints influence testing priorities, defect prevention, and overall system reliability.

Q4. Scenario: Smart Library Management System

Features:

Book Search, Book Reservation, Borrow/Return, Notifications

Usability Constraints:

- Search results displayed within 2 seconds
- Accessible interface for all age groups
- Minimal learning curve for first-time users

Question:

Conduct a usability evaluation using appropriate testing methods. Analyze how the given constraints impact interface design and recommend improvements to enhance usability and accessibility.

Q5. Scenario: Airline Reservation System

Modules:

User Login, Flight Search, Seat Selection, Ticket Booking

QA & Usability Constraints:

- No booking duplication
- System must support peak holiday traffic
- Booking interface should be simple and error-free

Question:

Apply both quality assurance and usability testing techniques under the specified constraints. Identify major risks, propose suitable testing methods, and explain how the system can be optimized for reliability, performance, and user experience.

Constraint-Based Problem Solving: QA & Usability Testing Answers

Q1. Online Banking System

System Overview

Components: Account Management, Fund Transfer, Bill Payment, Transaction History

Constraints: Transaction time < 2 sec | Multi-factor authentication (MFA) | 5000+ concurrent users

Step 1: Constraint-Driven Risk Analysis

Constraint	Quality Attribute	Risk Area
< 2 sec transaction time	Performance	Timeouts, DB locking, slow API calls under load
MFA required	Security	Bypass, session hijacking, OTP replay attacks
5000+ concurrent users	Scalability/Reliability	Race conditions, deadlocks, server crashes

Step 2: Potential Defects

Security defects

- Session tokens not invalidated after MFA timeout, allowing session fixation.
- OTP/SMS code reusable within its validity window (replay attack).
- Fund transfer API accepting requests without re-validating the second factor (MFA bypass on retry).
- Sensitive data (account numbers) exposed in logs or unencrypted API responses.

Performance defects

- Fund transfer exceeding 2 seconds when database has row-level locking during concurrent writes.
- Transaction History queries slowing down as record volume grows (missing indexing).
- API gateway or authentication server becoming a bottleneck under peak load.

Reliability defects

- Duplicate fund transfers if the "submit" request is retried after a timeout (no idempotency key).
- Partial transaction commits (money debited but not credited) during server failure — lack of atomicity.
- Session state lost when load balancer routes a user's MFA steps to different servers.

Step 3: Testing Strategies & Justification

Strategy	Purpose	Justification
Security Testing (penetration testing, OWASP ZAP/Burp Suite scans)	Detect MFA bypass, session hijacking, SQL injection	Banking systems handle sensitive financial data; security flaws have legal/financial consequences
Load & Stress Testing (JMeter, LoadRunner)	Simulate 5000+ concurrent users	Validates the system meets throughput and the 2-second SLA under real-world peak conditions
Performance Profiling	Identify slow DB queries, API bottlenecks	Ensures the <2 sec constraint is met consistently, not just on average
Reliability/Failover Testing	Kill server mid-transaction, test recovery	Confirms ACID compliance (atomicity) so money is never lost or duplicated
Idempotency Testing	Resend identical transfer requests	Prevents duplicate debits from network retries
Concurrency/Race Condition Testing	Simultaneous transfers from same account	Prevents overdraft or balance corruption
Regression Testing (automated)	Re-verify MFA and transfer logic after each release	Ensures new features don't reintroduce security/performance defects

Conclusion: Combining security testing (to protect MFA integrity), performance/load testing (to guarantee the 2-second SLA at 5000+ users), and reliability/failover testing (to guarantee transactional consistency) directly maps each QA technique to a stated constraint, ensuring the system is secure, fast, and dependable.

Q2. E-Commerce Shopping Application

System Overview

Features: Product Search, Shopping Cart, Checkout, Payment Gateway

Constraints: Checkout ≤ 3 steps | Easy navigation | Responsive design (mobile + desktop)

Step 1: Usability Testing Techniques Applied

Technique	What It Evaluates
Heuristic Evaluation (Nielsen's 10 heuristics)	Navigation clarity, error prevention, consistency
Task-Based Usability Testing	Can a real user complete checkout in ≤3 steps?
A/B Testing	Compare two checkout/navigation designs for conversion rate
Responsive/Cross-Device Testing	Layout, touch-target size, readability across screen sizes
Eye-Tracking / Click-Heatmaps	Identify where users struggle or drop off during search/checkout
System Usability Scale (SUS) Survey	Quantify overall user satisfaction post-task

Step 2: Usability Challenges Identified

- Checkout exceeding 3 steps** — separate pages for address, shipping method, and payment often push it to 4–5 steps.
- Poor product navigation** — deep category nesting or unclear filters increase search time and cognitive load.
- Inconsistent responsiveness** — cart icons/buttons too small on mobile; layout breaks on tablets (unhandled breakpoints).
- High cart abandonment** — hidden costs (shipping/tax) revealed only at the final step.
- Payment gateway friction** — redirect to a third-party page breaks visual continuity and trust.

Step 3: Recommended Design Improvements

Challenge	Improvement	Satisfies Constraint
Checkout too long	Merge address + shipping into one step; use guest checkout	≤3-step checkout
Poor navigation	Add faceted filters, autocomplete search, breadcrumb trails	Easy navigation
Inconsistent mobile layout	Use responsive grid (CSS Flexbox/Grid), touch-friendly 44px+ buttons	Responsive design
Hidden costs	Show real-time price/tax breakdown before final step	Reduces abandonment
Payment friction	Embed payment gateway inline (no redirect), support wallets (UPI/Apple Pay)	Faster, trustworthy checkout

Step 4: Impact on Customer Satisfaction & Efficiency

- A **3-step checkout** reduces task completion time and cognitive load, directly increasing conversion rates.
- **Consistent responsive design** removes friction across devices, widening the reachable user base.
- **Simplified navigation** shortens the search-to-purchase path, improving perceived efficiency.
- Higher **SUS scores** post-improvement validate increased satisfaction, which correlates with repeat purchases and lower cart abandonment.

Q3. Hospital Management System (BL5)

System Overview

Modules: Patient Registration, Appointment Scheduling, Medical Records, Billing

Constraints: Data accuracy | 24/7 availability | Simultaneous multi-department access

Step 1: Constraint-to-Priority Mapping

Constraint	QA Priority	Reason
Data accuracy	Critical (P0)	Wrong medical data can cause misdiagnosis or wrong treatment — patient safety risk
24/7 availability	Critical (P0)	Emergency care cannot wait for system downtime
Simultaneous access	High (P1)	Multiple departments (nurse, doctor, billing) editing the same record risks conflicts

Step 2: Critical Test Cases

Patient Registration

- Verify duplicate patient records are not created (unique ID/Aadhaar/SSN validation).
- Validate mandatory fields (allergies, blood group) cannot be left blank.

Appointment Scheduling

- Verify no double-booking of the same doctor/time slot under concurrent requests.

Medical Records

- Verify record updates by one department (e.g., lab result) are instantly visible to another (doctor) — data consistency.
- Test audit trail: every edit is logged with timestamp and user ID (traceability/accountability).
- Test data encryption and role-based access (nurse vs. doctor vs. billing should see different fields).

Billing

- Verify billing amounts sync correctly with treatment/medicine records (no mismatch).
- Test billing continuation during a partial system failure (no lost transactions).

Availability/Failover

- Simulate server crash — verify failover to backup server with zero/minimal downtime.
- Test database replication and backup restoration (disaster recovery drill).

Step 3: How Constraints Shape Testing Strategy

1. Data accuracy → Defect Prevention Focus

- Use **static testing/code reviews** and **input validation testing** early (shift-left) to catch data-entry defects before they reach production.
- Apply **boundary value & data integrity testing** on all patient data fields.
- Use **database constraint testing** (foreign keys, triggers) to prevent orphaned/inconsistent records.

2. 24/7 availability → Reliability-Centered Testing

- **Failover and disaster recovery testing** becomes mandatory, not optional.
- **Soak testing** (extended-duration load) to detect memory leaks that cause crashes over time.
- **Redundancy testing** for backup servers and database replicas.

3. Simultaneous multi-department access → Concurrency Testing Priority

- **Concurrency/race-condition testing** for shared records (e.g., simultaneous nurse + doctor updates).
- **Role-based access control (RBAC) testing** to prevent unauthorized cross-department edits.

Step 4: Overall System Reliability

By prioritizing **data-integrity tests** and **failover/concurrency tests** as P0/P1, defects that could compromise patient safety or system uptime are caught before release. This risk-based prioritization (driven directly by the stated constraints) ensures the QA effort is concentrated where failure has the highest real-world cost — a hallmark of mature, evaluative (BL5) QA judgment.

Q4. Smart Library Management System

System Overview

Features: Book Search, Book Reservation, Borrow/Return, Notifications

Constraints: Search ≤ 2 sec | Accessible for all age groups | Minimal learning curve

Step 1: Usability Evaluation Methods

Method	Application
Performance-based Usability Testing	Measure actual search response time across catalog sizes
Cognitive Walkthrough	Simulate a first-time/elderly user's mental steps to reserve a book
Accessibility Audit (WCAG 2.1 compliance check)	Evaluate contrast, font size, screen-reader compatibility, keyboard navigation
Think-Aloud Protocol	Observe users of different age groups verbalizing confusion points
Card Sorting	Test if book categorization matches users' mental models

Step 2: Impact of Constraints on Interface Design

- Search results within 2 seconds
 - Forces **indexed/optimized search backend** (e.g., Elasticsearch) instead of full-table scans.
 - Limits use of overly complex live-filtering UI that triggers repeated slow queries — requires debounced search input.
- Accessible interface for all age groups
 - Requires **larger font sizes, high-contrast themes, voice search/screen-reader support** for elderly or visually impaired users.
 - Simple iconography with text labels (not icon-only) to serve users unfamiliar with app conventions.
- Minimal learning curve
 - Drives a **flat navigation structure** (few clicks to any function) instead of nested menus.
 - Encourages **familiar UI patterns** (search bar at top, cart-like "reservation basket") that users already know from other apps.

Step 3: Usability Challenges Found (typical evaluation outcomes)

- Search feels slow when filters (author + genre + availability) are applied together.
- Elderly users struggle with small tap targets and icon-only buttons on the reservation screen.
- First-time users don't understand notification icons (return-due reminders) without labels.
- Multi-step borrow/return confirmation causes drop-off for less tech-savvy users.

Step 4: Recommended Improvements

Issue	Recommendation
Slow filtered search	Add search indexing + caching; show results incrementally (progressive loading)
Small tap targets	Increase button size ($\geq 44\text{px}$), add text labels next to icons
Confusing notifications	Use plain-language alerts ("Book due in 2 days") instead of icons alone
Multi-step confirmation	Reduce borrow/return to a single-tap confirmation with undo option
Age-inclusive accessibility	Add adjustable font-size control, voice-assist search, high-contrast mode toggle

Step 5: Outcome

These changes directly satisfy all three constraints: **optimized backend** + **progressive UI** maintains the 2-second search limit even as improvements are added; **accessibility features** (font scaling, labels, contrast) serve all age groups; and a **flatter, familiar navigation** keeps the learning curve minimal — together improving both usability and long-term adoption.

Q5. Airline Reservation System (BL4)

System Overview

Modules: User Login, Flight Search, Seat Selection, Ticket Booking

Constraints: No booking duplication | Support peak holiday traffic | Simple, error-free booking interface

Step 1: Major Risks Identified

Risk	Category	Description
Double booking of the same seat	Reliability/Data Integrity	Two users select and confirm the same seat simultaneously
System crash during peak load (holidays)	Performance/Scalability	Sudden spike in concurrent searches/bookings overwhelms servers
Payment success but booking failure	Reliability	Money deducted, but seat/ticket not generated due to a mid-transaction failure
Confusing seat-selection UI	Usability	Users select the wrong seat/class, leading to booking errors and support tickets
Session timeout during multi-step booking	Usability/Reliability	User loses progress on slow networks, causing repeated attempts (compounding duplication risk)

Step 2: Testing Methods Proposed

Quality Assurance (Functional/Non-Functional) Testing

- **Concurrency Testing:** Simulate multiple users booking the same seat at the same instant → verify only one succeeds and others get a real-time "seat unavailable" response.
- **Load & Stress Testing:** Simulate holiday-peak traffic (e.g., 10x normal volume) using tools like JMeter to identify the breaking point and validate auto-scaling.
- **Transaction/Idempotency Testing:** Resend the same booking request (simulating network retry) to confirm no duplicate ticket/charge is created.
- **Failover Testing:** Kill the booking service mid-transaction to verify rollback (payment refunded if seat isn't confirmed) — ensures atomicity.
- **Database Locking/Integrity Testing:** Verify row-level locks on seat inventory prevent race conditions.

Usability Testing

- **Task-Based Testing:** Time users completing login → search → seat selection → booking; flag steps causing errors or hesitation.
- **Error-Prevention Heuristic Review:** Check that the UI confirms destructive/critical actions (e.g., "Confirm seat 14A – Window") before final submission.
- **Accessibility & Cross-Device Testing:** Ensure the booking flow works identically and clearly on mobile apps and desktop browsers during high-traffic events.

Step 3: Optimization Recommendations

Goal	Optimization
Reliability (no duplication)	Implement optimistic/pessimistic locking on seat inventory; use unique idempotency keys per booking request; two-phase commit between payment and booking services
Performance (peak traffic)	Horizontal auto-scaling, caching flight search results, queueing system (virtual waiting room) for extreme spikes like holiday sales
User Experience (simple, error-free)	Real-time seat-map availability updates (no stale data), inline validation, clear progress indicator across booking steps, session auto-save to prevent data loss

Step 4: Justification

- **Concurrency and idempotency testing** directly target the "no duplication" constraint by proving the system handles simultaneous requests safely.
- **Load/stress testing** validates the system architecturally supports peak holiday traffic before it's exposed to real users.
- **Usability testing combined with error-prevention design** ensures the "simple, error-free" constraint is met, reducing both user errors and the support/refund burden they create.

Together, these QA and usability strategies address the airline system holistically: **data integrity** (no duplicate bookings), **scalability** (peak traffic), and **user experience** (error-free simplicity) — the three pillars explicitly required by the constraints.

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided